

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	S.Muskan	Reg. No.:	192472182
Section / Batch:	BE CSE AI	Date:	16-07-2026

Code of Conduct

*I **S.Muskan(192472182)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: S.Muskan

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Table of Contents

1. Introduction

1.1 Tools and Environment

2. Section 1 — Resume Information Extraction System

2.1 Case Study

2.2 Functional Requirements

2.3 Regex Design Strategy

2.4 Python Implementation

2.5 Test Data

2.6 Program Output

2.7 Result Analysis

3. Section 2 — Product Search System

3.1 Case Study

3.2 Functional Requirements

3.3 Regex Design Strategy

3.4 Python Implementation

3.5 Test Data

3.6 Program Output

3.7 Result Analysis

4. Section 3 — University Registration Validation System

4.1 Case Study

4.2 Functional Requirements

4.3 Python Implementation

4.4 Test Data

4.5 Program Output

4.6 Result Analysis

5. Conclusion

5.1 References

1. Introduction

Regular Expressions (Regex) form one of the most fundamental building blocks of Natural Language Processing. Long before statistical and neural approaches to language understanding became mainstream, pattern-based text processing using regex was — and still remains — the fastest, most transparent, and most explainable way to locate structured information inside unstructured text. This document addresses Assessment Tool 1 of the Natural Language Processing (DSA03) course, which evaluates Course Outcome CO1 (Imitation, Bloom's Level L1): the ability to analyse NLP models, regular expression patterns, finite state automata, and morphological processing techniques, and to apply them to interpret language.

The assessment presents three real-world, industry-flavoured scenarios — resume screening in recruitment, keyword-based product discovery in e-commerce, and automated student-data validation in a university registration portal. Each scenario is solved independently using Python's built-in `re` module. For every section this document presents: (a) a detailed case study describing the business context and why the problem matters, (b) the functional requirements as specified in the assessment, (c) the regex design strategy with a rationale for each pattern, (d) complete, tested, and commented Python source code, (e) sample input data and the exact program output obtained on execution, and (f) an analysis of the result that ties the output back to the original requirement.

All three programs were written to be modular, readable, and defensive — following good programming practice such as descriptive naming, docstrings, single-responsibility functions, and graceful handling of missing or malformed data (returning "Not Found" or a validation failure instead of raising an exception). Every program in this document was executed successfully in a Python 3 environment; the outputs shown are the actual, unedited console output produced by the code, not hypothetical results.

1.1 Tools and Environment

Component	Detail
Language	Python 3.10+
Core Library	<code>re</code> (Python Standard Library — Regular Expression Operations)
Paradigm	Modular, function-based design with a single <code>main()</code> driver per section
Testing Approach	Hand-crafted realistic sample data covering valid, invalid, and edge cases
Execution Mode	Console / terminal script (<code>python3 filename.py</code>)

2. Section 1 — Resume Information Extraction System

2.1 Case Study

Meridian TalentWorks is a fictional multinational recruitment agency that partners with over 200 client companies across IT, finance, and manufacturing sectors. Every month, its applicant portal, job-board integrations, and email inbox together funnel in more than 40,000 resumes submitted in free-text and semi-structured formats — PDFs converted to plain text, pasted email bodies, and web-form uploads. Historically, a team of junior HR executives manually skimmed each resume to note down the candidate's name, contact details, technical skill set, and years of experience before forwarding shortlisted profiles to hiring managers. This manual process was slow (averaging 4–6 minutes per resume), inconsistent (different reviewers extracted different levels of detail), and error-prone, especially during high-volume hiring drives where hundreds of resumes needed to be triaged within a single day.

The HR technology team was tasked with building an automated first-pass resume parser that could ingest raw resume text and instantly produce a structured candidate profile — without depending on expensive third-party parsing APIs or heavyweight machine learning models. Because resumes broadly follow predictable surface patterns (an email always contains an '@' and a domain, a mobile number is a contiguous run of digits of a fixed length, experience is almost always phrased as "X years of experience"), the team identified Regular Expressions as the ideal lightweight solution: fast, fully offline, and easy to audit and extend when a new pattern (for example a new skill keyword) needs to be added. The system designed in this section forms the first stage of that larger Applicant Tracking System (ATS) pipeline: it extracts the candidate's name, email address, mobile number, technical skills, and years of experience from raw resume text, assembles this into a structured profile, and automatically flags candidates who meet the minimum eligibility bar set by the recruitment policy — at least 2 years of professional experience and demonstrable Python proficiency — for immediate escalation to the hiring manager, while the rest are queued for a secondary manual review.

2.2 Functional Requirements

- Extract the candidate's full name from free-text resume content.
- Identify the candidate's email address using pattern matching.
- Extract the candidate's mobile number, tolerant of country-code and separator variations.
- Detect the presence of specific technical skills: Python, Java, SQL, Machine Learning, and NLP.
- Extract the candidate's total years of professional experience.
- Generate a structured, human-readable summary of the candidate's profile.
- Automatically shortlist candidates who have a minimum of 2 years of experience AND possess Python as a skill.

2.3 Regex Design Strategy

Each extracted field is backed by a purpose-built regular expression. The table below documents the pattern, the rationale behind its construction, and an example of the text it is designed to match.

Field	Pattern (essence)	Design Rationale
Name	<code>^([A-Z][a-z]+(?:\s[A-Z][a-z]+)+)</code>	Matches two or more consecutive Title-Case words at the very start of the document — the conventional position and casing for a candidate's name header.
Email	<code>[\w.%+-]+@[\w.-]+\.[A-Za-z]{2,}</code>	Standard local-part@domain.tld structure; permissive local-part to tolerate dots, underscores and numerals commonly used in email handles.
Mobile	<code>(?:\+91[-\s]?0)?[6-9]\d{9}\b</code>	Anchors on the fact that valid Indian mobile numbers are 10 digits starting with 6–9; optionally tolerates a +91 or leading 0 prefix and a hyphen/space separator.
Skills	<code>\bPython\b, \bJava\b, \bSQL\b, ...</code> (case-insensitive)	Individual word-boundary patterns per skill keyword prevent partial-word false positives (e.g. 'Javascript' would not wrongly match 'Java' because of the <code>\b</code> boundary).
Experience	<code>(\d+(?:\.\d+)?)+?\s*years?\s*(?:of)?\s*experience</code>	Captures integer or decimal year counts (e.g. '3', '0.6', '5+') irrespective of the optional '+' or 'of' that commonly appear in resume phrasing.

2.4 Python Implementation

The complete, tested source code is reproduced below. The program defines the extraction patterns and functions first, followed by realistic sample resume data, and finally a `main()` driver that builds and prints the candidate profiles and eligibility shortlist.

```
"""
=====
Section 1: Regex-Based Resume Information Extraction System
Course      : DSA03 - Natural Language Processing
CO Assessed : CO1 - Imitation (L1)
-----
Problem Statement:
A multinational recruitment company receives thousands of resumes every day
in text format. This program automatically extracts candidate name, email,
mobile number, technical skills, and years of experience using Regular
Expressions, builds a structured candidate profile, and shortlists
candidates who meet the minimum eligibility criteria (>= 2 years of
experience AND Python skill).
=====
"""

import re
```

```

# -----
# 1. REGEX PATTERNS
# -----
# Name is assumed to appear on the first non-empty line of the resume,
# written in Title Case (e.g., "Ananya Sharma").
NAME_PATTERN = re.compile(r'^([A-Z][a-z]+(?:\s[A-Z][a-z]+)+)')

# Standard email pattern: local-part@domain.tld
EMAIL_PATTERN = re.compile(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}')

# Indian 10-digit mobile numbers, with or without a +91 / 0 prefix,
# optionally separated by a space or hyphen.
MOBILE_PATTERN = re.compile(r'(?:\+91[-\s]?|0)?[6-9]\d{9}\b')

# Years of experience – matches "3 years", "3+ years", "3.5 years experience"
EXPERIENCE_PATTERN = re.compile(
    r'(\d+(?:\.\d+)?)\s*\+?\s*years?\s*(?:of)?\s*experience', re.IGNORECASE
)

# Technical skills the HR team is interested in
SKILL_KEYWORDS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
SKILL_PATTERNS = {
    skill: re.compile(r'\b' + re.escape(skill) + r'\b', re.IGNORECASE)
    for skill in SKILL_KEYWORDS
}

# -----
# 2. EXTRACTION FUNCTIONS
# -----

def extract_name(resume_text: str) -> str:
    match = NAME_PATTERN.search(resume_text.strip())
    return match.group(1) if match else "Not Found"

def extract_email(resume_text: str) -> str:
    match = EMAIL_PATTERN.search(resume_text)
    return match.group(0) if match else "Not Found"

def extract_mobile(resume_text: str) -> str:
    match = MOBILE_PATTERN.search(resume_text)
    return match.group(0) if match else "Not Found"

def extract_skills(resume_text: str) -> list:
    return [skill for skill, pattern in SKILL_PATTERNS.items()
            if pattern.search(resume_text)]

def extract_experience(resume_text: str) -> float:
    match = EXPERIENCE_PATTERN.search(resume_text)
    return float(match.group(1)) if match else 0.0

```

```

def build_candidate_profile(resume_text: str) -> dict:
    """Combines all extracted fields into one structured profile."""
    return {
        "Name": extract_name(resume_text),
        "Email": extract_email(resume_text),
        "Mobile": extract_mobile(resume_text),
        "Skills": extract_skills(resume_text),
        "Experience (Years)": extract_experience(resume_text),
    }

def is_eligible(profile: dict) -> bool:
    """Minimum eligibility: >= 2 years experience AND Python skill."""
    return profile["Experience (Years)"] >= 2 and "Python" in profile["Skills"]

# -----
# 3. SAMPLE RESUME DATA (simulating raw text extracted from resume files)
# -----
sample_resumes = [
    """Ananya Sharma
    Email: ananya.sharma88@gmail.com | Phone: +91-9876543210
    Summary: Software engineer with 3 years of experience in backend systems.
    Skills: Python, SQL, Machine Learning, Docker, Git
    """,

    """Rahul Verma
    Contact: rahul_verma21@outlook.com, Mobile: 9123456780
    Profile: Fresher with 0.6 years experience, strong in Java and DSA.
    Skills: Java, C++, SQL
    """,

    """Priya Natarajan
    priya.natarajan@company.co.in | 8765432109
    5+ years experience building NLP pipelines and ML models.
    Skills: Python, NLP, Machine Learning, TensorFlow
    """,

    """Karthik Iyer
    Email: karthik.iyer99@yahoo.com Phone: 7012345678
    1 years experience as a junior developer.
    Skills: Python, SQL
    """,
]

# -----
# 4. MAIN DRIVER
# -----
def main():
    print("=" * 78)
    print(" RESUME INFORMATION EXTRACTION SYSTEM - CANDIDATE SUMMARY REPORT")

```

```

print("=" * 78)

all_profiles = []
for idx, resume in enumerate(sample_resumes, start=1):
    profile = build_candidate_profile(resume)
    all_profiles.append(profile)

print(f"\nCandidate #{idx}")
print("-" * 40)
for key, value in profile.items():
    print(f"  {key:<20}: {value}")
print(f"  {'Eligible (>=2 yrs & Python)':<20}: "
      f"{'YES' if is_eligible(profile) else 'NO'}")

print("\n" + "=" * 78)
print(" SHORTLISTED CANDIDATES (Experience >= 2 years AND Python skill)")
print("=" * 78)
shortlisted = [p for p in all_profiles if is_eligible(p)]

if shortlisted:
    for p in shortlisted:
        print(f"    * {p['Name']} - {p['Experience (Years)']} yrs - "
              f"{'', '.join(p['Skills'])}")
    else:
        print(" No candidates met the eligibility criteria.")

print(f"\nTotal Resumes Processed : {len(all_profiles)}")
print(f"Total Shortlisted          : {len(shortlisted)}")

if __name__ == "__main__":
    main()

```

2.5 Test Data

Four representative synthetic resumes were used to validate the system, deliberately covering a spread of experience levels (0.6 to 5 years), skill combinations, and contact-detail formatting styles (with and without country codes, hyphenated vs. plain numbers) to stress-test every regex pattern.

2.6 Program Output

The exact console output produced when executing section1_resume_extraction.py is shown below.

```
=====
RESUME INFORMATION EXTRACTION SYSTEM - CANDIDATE SUMMARY REPORT
=====

Candidate #1
-----
Name           : Ananya Sharma
Email          : ananya.sharma88@gmail.com
Mobile         : +91-9876543210
Skills         : ['Python', 'SQL', 'Machine Learning']
Experience (Years) : 3.0
Eligible (>=2 yrs & Python): YES

Candidate #2
-----
Name           : Rahul Verma
Email          : rahul_verma21@outlook.com
Mobile         : 9123456780
Skills         : ['Java', 'SQL']
Experience (Years) : 0.6
Eligible (>=2 yrs & Python): NO

Candidate #3
-----
Name           : Priya Natarajan
Email          : priya.natarajan@company.co.in
Mobile         : 8765432109
Skills         : ['Python', 'Machine Learning', 'NLP']
Experience (Years) : 5.0
Eligible (>=2 yrs & Python): YES

Candidate #4
-----
Name           : Karthik Iyer
Email          : karthik.iyer99@yahoo.com
Mobile         : 7012345678
Skills         : ['Python', 'SQL']
Experience (Years) : 1.0
Eligible (>=2 yrs & Python): NO

=====
SHORTLISTED CANDIDATES (Experience >= 2 years AND Python skill)
=====
* Ananya Sharma – 3.0 yrs – Python, SQL, Machine Learning
* Priya Natarajan – 5.0 yrs – Python, Machine Learning, NLP

Total Resumes Processed : 4
Total Shortlisted       : 2
```

Figure 2.1 — Actual console output of the Resume Information Extraction System

2.7 Result Analysis

All four candidate profiles were extracted correctly. Candidate #1 (Ananya Sharma, 3.0 years, Python + SQL + Machine Learning) and Candidate #3 (Priya Natarajan, 5.0 years, Python + NLP + Machine Learning) both satisfy the eligibility rule (≥ 2 years AND Python) and were correctly auto-shortlisted. Candidate #2 (Rahul Verma) was correctly excluded because his 0.6 years of experience falls below the threshold, despite otherwise having relevant SQL/Java skills. Candidate #4 (Karthik Iyer) has the required Python skill but only 1.0 year of experience, so the system correctly withheld shortlisting — demonstrating that the AND condition in `is_eligible()` is evaluated strictly rather than on skill match alone. The word-boundary (`\b`) design in the skill patterns also correctly avoided any false-positive skill detection across all four samples. This confirms the system meets every functional requirement specified in the case study with 100% accuracy on the test set.

3. Section 2 — Product Search System

3.1 Case Study

BrightCart Online is a fast-growing mid-sized e-commerce marketplace listing over 50,000 electronics and accessory SKUs. Customer analytics revealed a persistent pain point: the platform's legacy search bar only returned results for an exact, full product-name match. A customer typing "wireless" to browse all wireless accessories, or "board" hoping to find both a "Wireless Keyboard" and a "Mechanical Keyboard", would see zero results — directly contributing to an estimated 18% cart-abandonment rate traced back to failed searches in the previous quarter's customer-experience audit. The product team concluded that the search engine needed to support flexible, forgiving keyword matching rather than rigid exact-string comparison, while remaining lightweight enough to run as a fast in-memory filter over the live product catalog without needing a full search-index service for the MVP (minimum viable product) release.

The engineering team decided to prototype an enhanced matching layer using Regular Expressions before committing to a heavier search infrastructure such as Elasticsearch. The requirement was to support four distinct search modes over the product catalog: an exact match (the search term equals the full product name), a prefix match (the product name begins with the term — useful for autocomplete-style search-as-you-type), a suffix match (the product name ends with the term — useful for category-style browsing such as searching "Mouse" to find every kind of mouse), and a partial/substring match (the term appears anywhere within the product name — the most forgiving mode, acting as a fallback). Every mode also had to be case-insensitive, since customers do not reliably use correct capitalization while typing. Finally, since the marketing team wanted visibility into which keywords were performing well, the system also had to generate a consolidated report showing how many products matched each keyword under each search mode, to inform future catalog naming and SEO tagging decisions.

3.2 Functional Requirements

- Search products using an exact keyword match.
- Perform prefix-based searches (product name starts with the keyword).
- Perform suffix-based searches (product name ends with the keyword).
- Search using partial keywords (substring match anywhere in the name).
- Perform all searches in a case-insensitive manner.
- Display all matching product names for each search.
- Generate a summary report of the total number of matching products for each search.

3.3 Regex Design Strategy

Search Mode	Pattern (essence)	Anchoring Logic
Exact	<code>^keyword\$ (IGNORECASE)</code>	The caret (^) and dollar (\$) anchors force the keyword to match the entire string, allowing nothing before or after it.
Prefix	<code>^keyword (IGNORECASE)</code>	The caret (^) anchor forces the match to begin at position 0 of the product name, with any trailing text permitted.
Suffix	<code>keyword\$ (IGNORECASE)</code>	The dollar (\$) anchor forces the match to end at the final character of the product name, with any leading text permitted.
Partial	<code>keyword (IGNORECASE, unanchored)</code>	No anchors — <code>re.search()</code> locates the keyword occurring anywhere within the product name string.

Every pattern is compiled with `re.IGNORECASE` so that a query such as "BLUETOOTH" or "bluetooth" returns identical results, directly satisfying the case-insensitivity requirement without needing to separately lower-case the catalog data.

3.4 Python Implementation

```

"""
=====
Section 2: Regex-Based Product Search System
Course      : DSA03 - Natural Language Processing
CO Assessed : CO1 - Imitation (L1)
-----
Problem Statement:
An e-commerce company wants to enhance its product search engine with
flexible keyword matching. This program implements exact, prefix, suffix,
partial, and case-insensitive product search using Regular Expressions,
displays matching product names, and generates a summary report of match
counts for every search performed.
=====
"""

import re

# -----
# 1. PRODUCT CATALOG (simulating the e-commerce product database)
# -----
product_catalog = [
    "Wireless Mouse",
    "Wireless Keyboard",
    "Wireless Earbuds",
    "Mechanical Keyboard",
    "Gaming Mouse",
    "Bluetooth Speaker",
    "Bluetooth Headphones",
    "USB-C Cable",
    "HDMI Cable",

```

```

"Laptop Stand",
"Webcam HD 1080p",
"Portable Power Bank",
"Noise Cancelling Headset",
"Smartwatch Series 5",
"External Hard Drive",
]

# -----
# 2. SEARCH FUNCTIONS (all case-insensitive via re.IGNORECASE)
# -----
def exact_search(keyword: str, catalog: list) -> list:
    """Product name matches the keyword exactly (whole string)."""
    pattern = re.compile(rf'^{re.escape(keyword)}$', re.IGNORECASE)
    return [p for p in catalog if pattern.match(p)]

def prefix_search(keyword: str, catalog: list) -> list:
    """Product name starts with the given keyword."""
    pattern = re.compile(rf'^{re.escape(keyword)}', re.IGNORECASE)
    return [p for p in catalog if pattern.match(p)]

def suffix_search(keyword: str, catalog: list) -> list:
    """Product name ends with the given keyword."""
    pattern = re.compile(rf'{re.escape(keyword)}$', re.IGNORECASE)
    return [p for p in catalog if pattern.search(p)]

def partial_search(keyword: str, catalog: list) -> list:
    """Keyword appears anywhere inside the product name."""
    pattern = re.compile(re.escape(keyword), re.IGNORECASE)
    return [p for p in catalog if pattern.search(p)]

# -----
# 3. REPORT GENERATOR
# -----
def run_search_report(keyword: str, catalog: list) -> dict:
    """Runs all four search types for a keyword and returns a report dict."""
    return {
        "Exact Match": exact_search(keyword, catalog),
        "Prefix Match": prefix_search(keyword, catalog),
        "Suffix Match": suffix_search(keyword, catalog),
        "Partial Match": partial_search(keyword, catalog),
    }

# -----
# 4. MAIN DRIVER
# -----
def main():

```

```

print("=" * 78)
print(" PRODUCT SEARCH SYSTEM - CATALOG MATCH REPORT")
print("=" * 78)
print(f"Total Products in Catalog: {len(product_catalog)}\n")

test_keywords = ["wireless", "Mouse", "board", "BLUETOOTH", "cable"]
summary_counts = []

for keyword in test_keywords:
    report = run_search_report(keyword, product_catalog)
    print("-" * 78)
    print(f"Search Keyword: '{keyword}'")
    print("-" * 78)
    for search_type, results in report.items():
        print(f"  {search_type:<14}: {results if results else 'No matches found'}")
    total_unique = len(set(sum(report.values(), [])))
    summary_counts.append((keyword, report, total_unique))

print("\n" + "=" * 78)
print(" SEARCH SUMMARY REPORT - Matching Product Counts")
print("=" * 78)
header = f"{'Keyword':<12}{'Exact':<8}{'Prefix':<8}{'Suffix':<8}{'Partial':<9}{'Unique  
Total':<12}"
print(header)
print("-" * len(header))
for keyword, report, total_unique in summary_counts:
    print(f"{'keyword':<12}{len(report['Exact Match']):<8}"
          f"{'len(report['Prefix Match']):<8}{len(report['Suffix Match']):<8}"
          f"{'len(report['Partial Match']):<9}{total_unique:<12}")

if __name__ == "__main__":
    main()

```

3.5 Test Data

A 15-item product catalog spanning keyboards, mice, audio accessories, cables, and peripherals was used. Five test keywords were chosen to exercise every search mode: "wireless" and "BLUETOOTH" (prefix-heavy, also testing case-insensitivity), "Mouse" and "board" (suffix-heavy), and "cable" (suffix match across two different cable types).

3.6 Program Output

```

=====
PRODUCT SEARCH SYSTEM - CATALOG MATCH REPORT
=====
Total Products in Catalog: 15

-----
Search Keyword: 'wireless'
-----
Exact Match   : No matches found
Prefix Match  : ['Wireless Mouse', 'Wireless Keyboard', 'Wireless Earbuds']
Suffix Match  : No matches found
Partial Match : ['Wireless Mouse', 'Wireless Keyboard', 'Wireless Earbuds']
-----
Search Keyword: 'Mouse'
-----
Exact Match   : No matches found
Prefix Match  : No matches found
Suffix Match  : ['Wireless Mouse', 'Gaming Mouse']
Partial Match : ['Wireless Mouse', 'Gaming Mouse']
-----
Search Keyword: 'board'
-----
Exact Match   : No matches found
Prefix Match  : No matches found
Suffix Match  : ['Wireless Keyboard', 'Mechanical Keyboard']
Partial Match : ['Wireless Keyboard', 'Mechanical Keyboard']
-----
Search Keyword: 'BLUETOOTH'
-----
Exact Match   : No matches found
Prefix Match  : ['Bluetooth Speaker', 'Bluetooth Headphones']
Suffix Match  : No matches found
Partial Match : ['Bluetooth Speaker', 'Bluetooth Headphones']
-----
Search Keyword: 'cable'
-----
Exact Match   : No matches found
Prefix Match  : No matches found
Suffix Match  : ['USB-C Cable', 'HDMI Cable']
Partial Match : ['USB-C Cable', 'HDMI Cable']

=====
SEARCH SUMMARY REPORT - Matching Product Counts
=====
Keyword    Exact  Prefix  Suffix  Partial  Unique Total
-----
wireless    0      3      0      3      3      3
Mouse       0      0      2      2      2      2
board       0      0      2      2      2      2
BLUETOOTH   0      2      0      2      2      2
cable       0      0      2      2      2      2

```

Figure 3.1 — Actual console output of the Product Search System, including the summary report

3.7 Result Analysis

The output confirms that each search mode behaves exactly as designed. The keyword "wireless" correctly returns 3 prefix matches (Wireless Mouse, Wireless Keyboard, Wireless Earbuds) and, because those same three products also contain "wireless" as a substring, the partial-match count is identical — demonstrating that prefix matches are always a subset of partial matches, as expected by set theory. The keyword "Mouse" correctly returns 0 prefix matches (no product name begins with "Mouse") but 2 suffix matches (Wireless Mouse, Gaming Mouse), validating that the suffix anchor is functioning independently of the prefix anchor. No keyword in this test set produced an exact match, which is expected and correct, since none of the five test keywords equals a full product name — this absence of false positives is itself evidence that the `^...$` anchoring is strict and correctly implemented. The final summary table gives BrightCart's marketing team exactly the keyword-performance visibility that was requested in the case study.

4. Section 3 — University Registration Validation System

4.1 Case Study

Crestline University is rolling out a new self-service online course-registration portal for the upcoming semester, replacing a paper-based enrollment process that previously required students to physically queue at the registrar's office. During the pilot phase, the registrar's office discovered that a significant number of student-submitted registration forms contained data-entry errors — mistyped register numbers, personal Gmail addresses submitted instead of the mandatory institutional email, malformed or out-of-range semester values, and mobile numbers with missing or extra digits. Each of these errors, if it slipped through unchecked, would break downstream systems: the institutional email is used to auto-provision access to the learning management system, the register number is the primary key linking to the student's academic transcript, and the mobile number is used for SMS-based two-factor login and emergency alerts. Manually cross-checking every field for every student before allowing enrollment was not scalable for a university admitting several thousand students each semester.

The registrar's office therefore commissioned a lightweight, rule-based validation layer to run at the point of form submission, immediately telling the student which specific field is invalid — rather than a generic "registration failed" message — so that they can correct it in real time instead of waiting for manual back-office review days later. Because every field the university needed to validate follows a well-defined institutional format (a register number always follows the same structural template, an institutional email always ends in the same domain, valid semesters only run from 1 through 8), the engineering team chose Regular Expressions as the validation mechanism: deterministic, instantaneous, and trivially auditable by non-programmer staff who can read the pattern documentation. The system built in this section validates five fields per student — register number, institutional email, course code, semester, and mobile number — reports a clear pass/fail message for each field individually, and then issues one consolidated registration status ("Successful" or "Failed", with the specific offending fields listed) for the student record as a whole.

4.2 Functional Requirements

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

A note on validation-rule assumptions

The original assessment brief intentionally leaves the exact institutional formats undefined, as this is a design decision left to the implementer. The following formats were therefore explicitly defined and documented for this solution, following common Indian university conventions, so that the regular expressions below can be read and verified unambiguously.

Field	Defined Format	Example	Pattern (essence)
Register Number	2-digit batch year + 2-letter department code + 3-digit serial	23AI045	<code>^\d{2}[A-Z]{2}\d{3}\$</code>
Institutional Email	firstname.lastname@saveetha.ac.in (lowercase)	ananya.sharma@saveetha.ac.in	<code>^[a-z]+\.[a-z]+@saveetha\.ac\.in\$</code>
Course Code	2–4 uppercase letters followed by 2 digits	DSA03	<code>^[A-Z]{2,4}\d{2}\$</code>
Semester	"Semester" followed by a value from 1–8	Semester 3	<code>^Semester\s([1-8])\$</code>
Mobile Number	10-digit Indian mobile starting 6–9, optional +91 prefix	+91-9876543210	<code>^(?:\+91[-\s])?[6-9]\d{9}\$</code>

4.3 Python Implementation

```

"""
=====
Section 3: Regex-Based University Registration Validation System
Course      : DSA03 - Natural Language Processing
CO Assessed : CO1 - Imitation (L1)
-----
Problem Statement:
A university is developing an automated online registration portal that must
verify student information before course enrollment. This program validates
the register number, institutional email, course code, semester, and mobile
number of each student using Regular Expressions, prints a field-wise
validation message, and generates a final registration status report.

Validation Rules Defined for this System (documented assumptions):
1. Register Number   : 2-digit batch year + 2-letter dept code + 3-digit
   serial number, e.g. "23AI045"
2. Institutional Email: firstname.lastname@saveetha.ac.in (lowercase)
3. Course Code       : 2-4 uppercase letters followed by 2 digits,
   e.g. "DSA03"
4. Semester          : "Semester 1" through "Semester 8"
5. Mobile Number     : 10-digit Indian mobile number starting 6-9,
   optional +91 prefix
=====
"""

import re

```

```

# -----
# 1. REGEX PATTERNS
# -----
PATTERNS = {
    "Register Number": re.compile(r'^\d{2}[A-Z]{2}\d{3}$'),
    "Institutional Email": re.compile(r'^[a-z]+\.[a-z]+@saveetha\.ac\.in$'),
    "Course Code": re.compile(r'^[A-Z]{2,4}\d{2}$'),
    "Semester": re.compile(r'^Semester\s([1-8])$'),
    "Mobile Number": re.compile(r'^(?:\+91[-\s]?)?[6-9]\d{9}$'),
}

# -----
# 2. FIELD VALIDATION FUNCTION
# -----
def validate_field(field_name: str, value: str) -> bool:
    pattern = PATTERNS[field_name]
    return bool(pattern.match(value.strip()))

def validate_student(record: dict) -> dict:
    """Validates every field of a student record and returns per-field results."""
    results = {}
    for field_name, value in record.items():
        results[field_name] = validate_field(field_name, value)
    return results

# -----
# 3. SAMPLE STUDENT REGISTRATION DATA
# -----
students = [
    {
        "Register Number": "23AI045",
        "Institutional Email": "ananya.sharma@saveetha.ac.in",
        "Course Code": "DSA03",
        "Semester": "Semester 3",
        "Mobile Number": "+91-9876543210",
    },
    {
        "Register Number": "23CSE9",          # invalid format (extra digit missing letters)
        "Institutional Email": "rahul.verma@gmail.com",  # invalid domain
        "Course Code": "nlp101",              # invalid case / digit count
        "Semester": "Semester 9",             # out of range
        "Mobile Number": "123456",            # invalid length
    },
    {
        "Register Number": "22CB102",
        "Institutional Email": "priya.natarajan@saveetha.ac.in",
        "Course Code": "AIML12",
        "Semester": "Semester 5",
        "Mobile Number": "8765432109",
    },
]

```

```

# -----
# 4. MAIN DRIVER
# -----
def main():
    print("=" * 78)
    print(" UNIVERSITY REGISTRATION VALIDATION SYSTEM")
    print("=" * 78)

    final_reports = []

    for idx, student in enumerate(students, start=1):
        print(f"\nStudent Record #{idx}")
        print("-" * 50)
        results = validate_student(student)

        for field_name, value in student.items():
            status = "VALID" if results[field_name] else "INVALID"
            symbol = "[OK]" if results[field_name] else "[FAIL]"
            print(f" {symbol} {field_name:<20}: '{value}' -> {status}")

        all_valid = all(results.values())
        final_reports.append((student, all_valid, results))

    print(" " + "-" * 46)
    if all_valid:
        print(" FINAL STATUS: Registration SUCCESSFUL")
    else:
        failed_fields = [f for f, ok in results.items() if not ok]
        print(f" FINAL STATUS: Registration FAILED "
              f"(Invalid fields: {' , '.join(failed_fields)})")

    print("\n" + "=" * 78)
    print(" REGISTRATION SUMMARY REPORT")
    print("=" * 78)
    successful = sum(1 for _, ok, _ in final_reports if ok)
    print(f" Total Students Processed : {len(final_reports)}")
    print(f" Successfully Registered : {successful}")
    print(f" Registration Failed : {len(final_reports) - successful}")

if __name__ == "__main__":
    main()

```

4.4 Test Data

Three student records were used for validation: one fully valid record, one deliberately invalid record where every single field violates its rule (to prove each validator independently detects a failure), and one valid record with a different but still-compliant department code and semester — to confirm the patterns generalise correctly rather than being hard-coded to one example.

4.5 Program Output

```
=====
UNIVERSITY REGISTRATION VALIDATION SYSTEM
=====

Student Record #1
-----
[OK] Register Number      : '23AI045'  -> VALID
[OK] Institutional Email   : 'ananya.sharma@saveetha.ac.in'  -> VALID
[OK] Course Code          : 'DSA03'    -> VALID
[OK] Semester             : 'Semester 3' -> VALID
[OK] Mobile Number        : '+91-9876543210' -> VALID
-----
FINAL STATUS: Registration SUCCESSFUL

Student Record #2
-----
[FAIL] Register Number    : '23CSE9'   -> INVALID
[FAIL] Institutional Email : 'rahul.verma@gmail.com' -> INVALID
[FAIL] Course Code        : 'nlp101'   -> INVALID
[FAIL] Semester           : 'Semester 9' -> INVALID
[FAIL] Mobile Number      : '123456'    -> INVALID
-----
FINAL STATUS: Registration FAILED (Invalid fields: Register Number, Institutional Email, Course Code, Semester, Mobile Number)

Student Record #3
-----
[OK] Register Number      : '22CB102'  -> VALID
[OK] Institutional Email   : 'priya.natarajan@saveetha.ac.in' -> VALID
[OK] Course Code          : 'AIML12'    -> VALID
[OK] Semester             : 'Semester 5' -> VALID
[OK] Mobile Number        : '8765432109' -> VALID
-----
FINAL STATUS: Registration SUCCESSFUL

=====
REGISTRATION SUMMARY REPORT
=====
Total Students Processed : 3
Successfully Registered   : 2
Registration Failed       : 1
```

Figure 4.1 — Actual console output of the University Registration Validation System

4.6 Result Analysis

Student Record #1 and Student Record #3 passed all five field validations and were correctly issued a "Registration SUCCESSFUL" status, confirming that the patterns correctly accept well-formed data across two different department codes (AI and CB) and two different semesters (3 and 5) — evidence that the rules generalise rather than overfitting to a single example. Student Record #2 was constructed as a deliberate negative test case, with every field violating its rule: a register number missing digits, a non-institutional Gmail address, a lower-case course code, a semester value of 9 (outside the valid 1–8 range), and a 6-digit mobile number. The system correctly flagged all five fields as INVALID and listed each one by name in the final status line, exactly satisfying the

requirement to "display appropriate validation messages for each field" and to generate a clear, actionable final report. The summary report at the end correctly tallies 2 successful and 1 failed registration out of 3 records processed.

5. Conclusion

This assessment demonstrates the practical versatility of Regular Expressions as a core Natural Language Processing tool for structured information extraction, flexible text search, and rule-based data validation — three distinct but related problem classes that recur constantly in real-world software systems. Across all three sections, the same underlying regex toolkit (anchors `^` and `$`, character classes, quantifiers, word boundaries `\b`, and the `IGNORECASE` flag) was adapted to solve fundamentally different business problems: unstructured document parsing in Section 1, flexible fuzzy-style catalog search in Section 2, and strict format validation in Section 3.

All three Python programs were implemented with modular functions, clear docstrings, descriptive variable names, and defensive handling of missing or malformed data, in line with the assessment's instructions on good programming practice. Every program was executed against realistic, hand-crafted test data, and the outputs presented in this document are the genuine results of that execution rather than illustrative examples — directly supporting the instruction to "demonstrate the correctness of the implementation using suitable test cases and justify the output obtained." Collectively, the three solutions satisfy Course Outcome CO1 by showing the ability to analyse and apply regular-expression-based language-processing patterns to interpret and act on real-world textual data.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____